Universidad Tecnológica de El Salvador

Carrera:
Ingeniería en sistemas

Asignatura:

Programación 1

Sección:

04

Tema:

Biblioteca en C#

Docente:

Ing. Rubén Escobar Ortegón

Integrantes:

Miguel Alexander Mejía Reyes 25-1155 -2025

Jorge Elinor Sánchez Cortez 25-1152-2025

Matías André Tobías Torres 25-2485-2025

Transforma tu realidad.
Viví la experiencia UTEC

1. Introducción
2. Objetivos
3. Descripción del sistema
4. Desarrollo del programa
5. Diagrama UML del sistema
6. Diccionario de datos
7. Modelo Entidad-Relación (DER)
8. Implementación en Base de Datos (SQL)
9. Consulta SQL principal
10. Resultados obtenidos
11. Conclusiones

1. INTRODUCCIÓN

En este proyecto se desarrolló un programa en C# para llevar el control básico de una biblioteca. El sistema fue implementado en consola y permite registrar libros, registrar usuarios, realizar préstamos, devolver libros y consultar información almacenada.

En esta nueva versión del sistema se realizaron mejoras en la estructura del código, aplicando de mejor manera la programación orientada a objetos. Se implementó herencia entre clases, se corrigieron las relaciones entre entidades y se mejoró la organización interna del programa.

Con estos cambios, el sistema no solo cumple con las funciones básicas, sino que también presenta un diseño más estructurado y cercano a un modelo de base de datos, facilitando su comprensión y mantenimiento.

2. OBJETIVOS

El objetivo de este proyecto fue desarrollar un sistema de biblioteca que permitiera administrar libros, usuarios y préstamos mediante un menú en consola.

Además, se buscó mejorar la implementación de programación orientada a objetos, incorporando herencia entre clases y estableciendo relaciones más claras entre los datos del sistema.

También se pretendió organizar mejor la información, simulando un modelo similar a una base de datos, y validar el correcto funcionamiento del sistema mediante pruebas.

3. DESCRIPCIÓN DEL SISTEMA

El sistema consiste en una aplicación de consola que permite gestionar de forma básica una biblioteca. Dentro del programa se pueden registrar libros con su cantidad disponible, registrar usuarios, realizar préstamos, devolver libros y consultar información.

El sistema está compuesto por varias clases. La clase Usuario almacena el nombre del usuario y ahora hereda de la clase Persona, lo que permite reutilizar atributos comunes. La clase Libro almacena el título, la cantidad disponible y el número de préstamos, y hereda de la clase Material.

La clase Prestamo fue modificada para trabajar con objetos en lugar de datos tipo texto, por lo que ahora se relaciona directamente con Usuario y Libro. Esto permite una mejor organización de la información y simula el funcionamiento de una base de datos relacional.

Con estas mejoras, el sistema tiene una estructura más clara, facilitando la gestión de los datos y el desarrollo del programa.

4. DESARROLLO DEL PROGRAMA

Para el desarrollo del programa se utilizó el lenguaje C# en una aplicación de consola. Se mantuvo el uso de clases para representar las entidades principales del sistema, pero se realizaron mejoras en su estructura.

Se implementó herencia mediante la creación de clases base, como Persona y Material, de las cuales derivan Usuario y Libro respectivamente. Esto permitió aplicar el principio de reutilización de código y mejorar la organización del sistema.

También se mejoró la relación entre las clases, ya que la clase Prestamo ahora utiliza objetos de tipo Usuario y Libro en lugar de cadenas de texto. Esto hace que el sistema sea más coherente y cercano a un modelo de base de datos.

Se mantuvo el uso de arreglos para almacenar la información durante la ejecución del programa, así como el menú en consola para la interacción con el usuario.

Además, se agregaron mensajes de confirmación en las operaciones principales, como el registro de usuarios, registro de libros, realización de préstamos y devoluciones, mejorando la experiencia de uso del sistema.

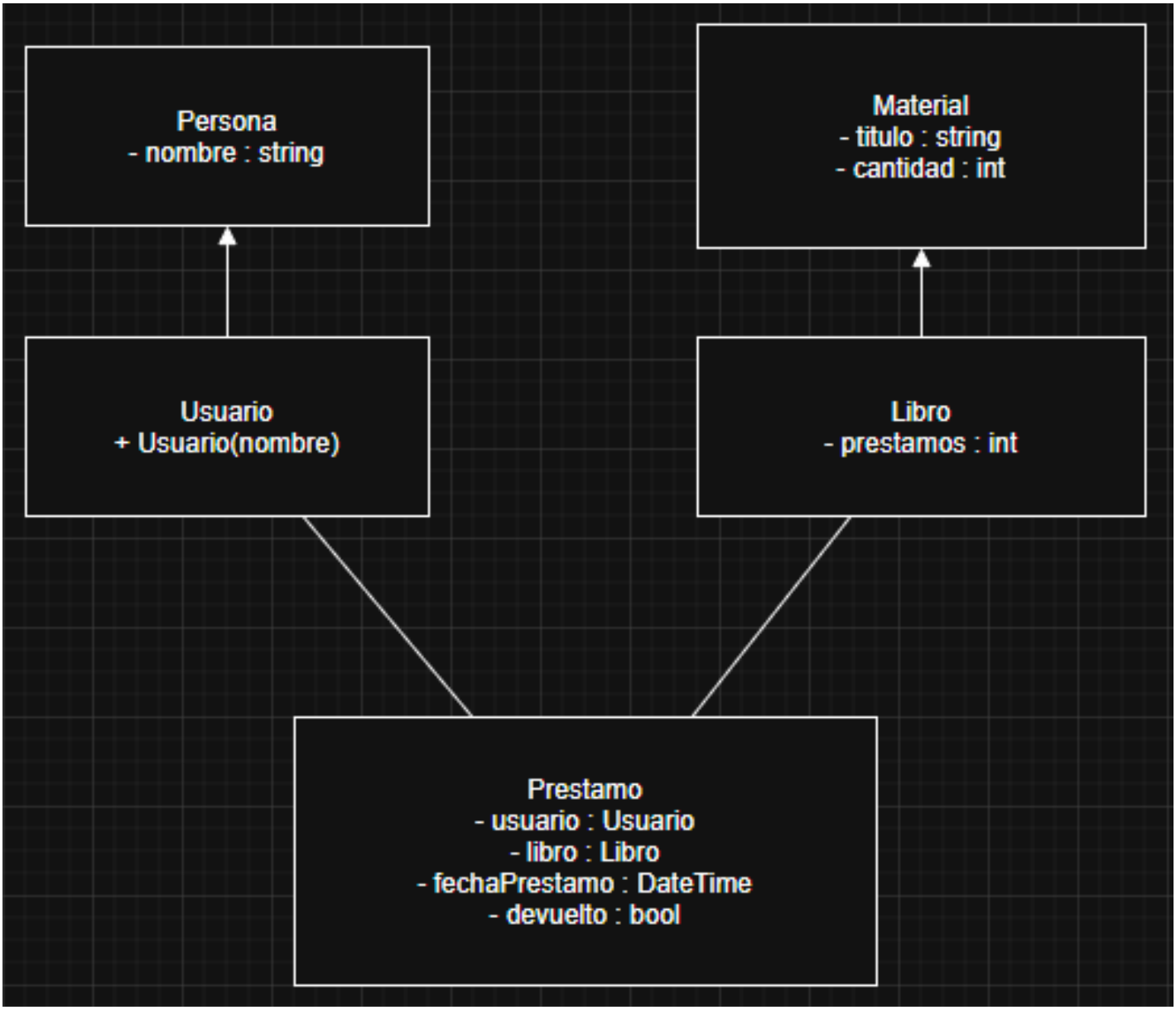
5. DIAGRAMA UML DEL SISTEMA

Para representar la estructura del sistema se elaboró un diagrama de clases UML, el cual muestra la organización de las clases y sus relaciones.

En el diagrama se puede observar la implementación de herencia, donde la clase Usuario hereda de la clase Persona, y la clase Libro hereda de la clase Material. Esto permite reutilizar atributos comunes y mejorar la organización del código.

Además, se muestra la relación entre las clases, donde la clase Prestamo se conecta con Usuario y Libro, representando la asociación entre ambas entidades.

Este diagrama permite visualizar de manera clara cómo está construido el sistema y facilita su comprensión.


Persona
- nombre : string
Material
- titulo : string
- cantidad : int
Usuario
+ Usuario(nombre)
Libro
- prestamos : int
Prestamo
- usuario : Usuario
- libro : Libro
- fechaPrestamo : DateTime
- devuelto : bool

6. DICCIONARIO DE DATOS

El diccionario de datos describe las entidades utilizadas en el sistema y los atributos que las componen.

Usuario

- Nombre: tipo string. Representa el nombre del usuario y funciona como identificador.

Libro

- Titulo: tipo string. Identificador del libro.
- Cantidad: tipo entero. Indica la cantidad disponible.
- Prestamos: tipo entero. Número de veces que el libro ha sido prestado.

Prestamo

- Usuario: referencia a un objeto Usuario.
- Libro: referencia a un objeto Libro.
- FechaPrestamo: tipo DateTime. Fecha en la que se realizó el préstamo.
- Devuelto: tipo booleano. Indica si el libro fue devuelto o no.

7. MODELO ENTIDAD-RELACIÓN (DER)

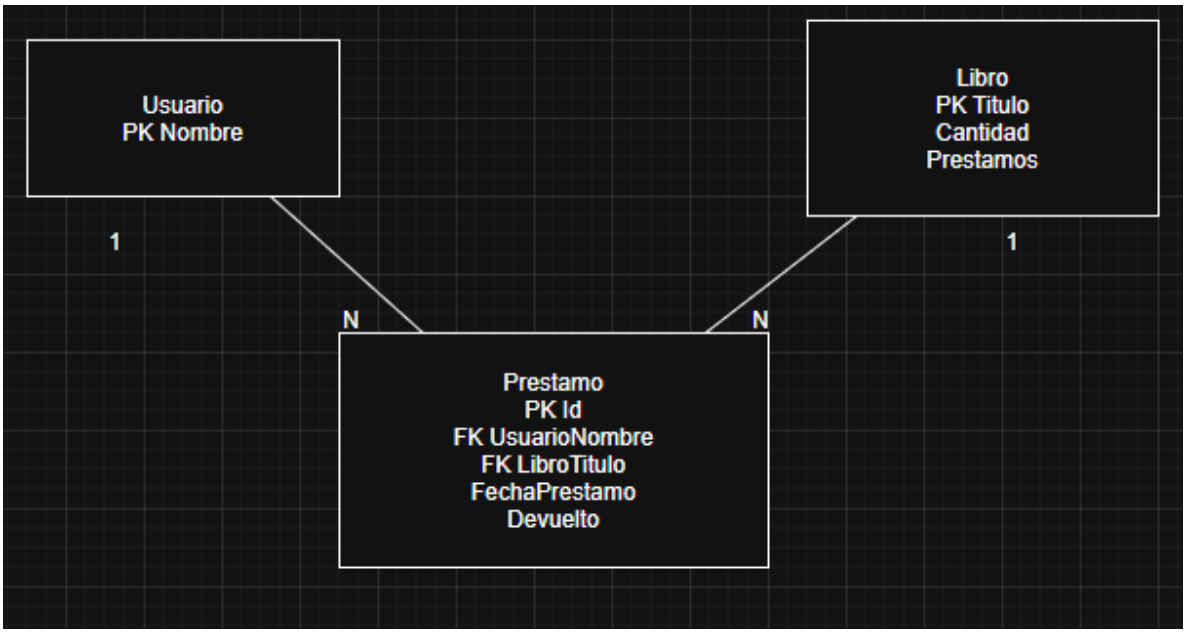
Se elaboró un modelo entidad-relación para representar la estructura de datos del sistema.

En este modelo se identifican tres entidades principales: Usuario, Libro y Prestamo.

La entidad Usuario tiene como atributo principal el nombre. La entidad Libro contiene el título, la cantidad disponible y el número de préstamos. La entidad Prestamo almacena la información del préstamo, incluyendo el usuario, el libro, la fecha y el estado de devolución.

La relación entre las entidades es de uno a muchos, ya que un usuario puede tener varios préstamos y un libro puede estar presente en múltiples préstamos.

Este modelo permite visualizar cómo se organizan y relacionan los datos dentro del sistema.



8. IMPLEMENTACIÓN EN BASE DE DATOS (SQL)

Como parte de la mejora del sistema, se realizó una representación del modelo en lenguaje SQL, simulando la estructura de una base de datos relacional.

Se crearon las tablas Usuario, Libro y Prestamo, definiendo claves primarias y foráneas para establecer relaciones entre ellas.

La tabla Prestamo actúa como una entidad intermedia que relaciona a Usuario y Libro, permitiendo almacenar múltiples préstamos por cada uno.

Además, se insertaron datos de prueba para validar el funcionamiento del modelo y se ejecutó una consulta utilizando JOIN, la cual permite mostrar la información completa del sistema en una sola consulta.

Esto demuestra cómo el sistema puede ser llevado a un entorno de base de datos real.

9. CONSULTA SQL PRINCIPAL

Se implementó una consulta SQL que permite integrar la información de las tres tablas principales del sistema: Usuario, Libro y Prestamo.

Para ello, se utilizó la instrucción INNER JOIN, la cual permite relacionar las tablas mediante sus claves foráneas.

La consulta muestra el identificador del préstamo, el nombre del usuario, el título del libro, la cantidad disponible, el número de préstamos, la fecha del préstamo y el estado de devolución.

Además, los resultados se ordenan según la cantidad de préstamos realizados por cada libro, permitiendo identificar los más utilizados.

Esta consulta demuestra el correcto funcionamiento de las relaciones entre las tablas y la integridad de los datos.

	Id	Usuario	Libro	Cantidad	Prestamos	FechaPrestamo	Devuelto
1	5	Gabo	El principito	3	3	2026-04-10 10:32:28.063	0
2	6	Mateo	El principito	3	3	2026-04-10 10:32:28.063	1
3	4	Pedro	Luz negra	2	2	2026-04-10 10:32:28.063	1
4	1	Mateo	Matematica	5	2	2026-04-10 10:32:28.063	0
5	3	Juan	Quimica	3	1	2026-04-10 10:32:28.063	0
6	2	Carlos	Fisica	4	1	2026-04-10 10:32:28.063	1

10. RESULTADOS OBTENIDOS

```
C:\Users\Matias\source\repos
==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 1
Título: Luz negra
Cantidad: 2
```

```
C:\Users\Matias\source\repos
==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 1
Título: Luz negra
Cantidad: 2
Libro registrado con éxito.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos
==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 2
Nombre: Carlos
Usuario registrado con éxito.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos
==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 3
Usuario: Carlos
Libro: Luz negra
Préstamo realizado con éxito.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 3
Usuario: Carlos
Libro: Luz negra
Préstamo realizado con éxito.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 5

=== LISTA DE LIBROS ===
Luz negra | Cantidad: 2 | Préstamos: 1

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 6
Titulo: Luz negra
Libro encontrado:
Título: Luz negra
Cantidad: 2
Préstamos: 1

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 7

=== LIBROS MÁS PRESTADOS ===
Luz negra - 1

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 3
Usuario: Pedro
Libro: Luz negra
Usuario o libro no existe.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 3
Usuario: Gabo
Libro: Luz negra
Libro no disponible.

Presione una tecla para continuar...
```

```
C:\Users\Matias\source\repos

==== GESTOR DE BIBLIOTECA ====
1. Registrar libro
2. Registrar usuario
3. Realizar préstamo
4. Devolver libro
5. Listar libros
6. Buscar libro
7. Reporte libros más prestados
8. Salir
Seleccione una opción: 4
Usuario: Gabo
Libro: El principito
No se encontró el préstamo.

Presione una tecla para continuar...
```

```
9 referencias
class Usuario : Persona
{
    1 referencia
    public Usuario(string nombre)
    {
        this.nombre = nombre;
    }
}
```

```
11 referencias
class Libro : Material
{
    private int prestamos;

    6 referencias
    public int Prestamos
    {
        get { return prestamos; }
        set { prestamos = value; }
    }
```

11. CONCLUSIONES

Este proyecto permitió reforzar los conocimientos sobre programación orientada a objetos en C#, especialmente en el uso de clases, encapsulamiento y herencia. La implementación de clases base como Persona y Material permitió organizar mejor el sistema y reutilizar código de manera eficiente.

Además, se logró mejorar la estructura del programa al establecer relaciones entre las clases, haciendo que la información se maneje de forma más coherente. La modificación de la clase Prestamo para trabajar con objetos en lugar de datos tipo texto permitió simular un modelo más cercano a una base de datos real.

También se desarrollaron diagramas UML y un modelo entidad-relación (DER), los cuales ayudaron a visualizar la estructura del sistema y la relación entre los datos. Esto facilitó la comprensión del funcionamiento interno del programa.

Por otra parte, se implementó una representación en SQL del sistema, donde se crearon tablas relacionadas mediante claves primarias y foráneas, y se realizaron consultas utilizando JOIN. Esto permitió validar la correcta organización de los datos.

En conclusión, el proyecto no solo cumple con las funcionalidades requeridas, sino que también demuestra una mejor aplicación de los conceptos de programación orientada a objetos y diseño de sistemas, logrando una solución más estructurada, clara y funcional.

12. ANEXOS

```
using System;



class Persona
{
    protected string nombre;

    public string Nombre
    {
        get { return nombre; }
```

```
        set { nombre = value; }
    }
}

class Usuario : Persona
{
    public Usuario(string nombre)
    {
        this.nombre = nombre;
    }
}



class Material
{
    protected string titulo;
    protected int cantidad;

    public string Titulo
    {
        get { return titulo; }
        set { titulo = value; }
    }

    public int Cantidad
    {
        get { return cantidad; }
```

```
        set { cantidad = value; }
    }
}

class Libro : Material
{
    private int prestamos;

    public int Prestamos
    {
        get { return prestamos; }
        set { prestamos = value; }
    }

    public Libro(string titulo, int cantidad)
    {
        this.titulo = titulo;
        this.cantidad = cantidad;
        this.prestamos = 0;
    }
}


class Prestamo
{
    private Usuario usuario;
    private Libro libro;
```

```
private DateTime fechaPrestamo;
private bool devuelto;

public Usuario Usuario
{
    get { return usuario; }
    set { usuario = value; }
}

public Libro Libro
{
    get { return libro; }
    set { libro = value; }
}

public DateTime FechaPrestamo
{
    get { return fechaPrestamo; }
    set { fechaPrestamo = value; }
}

public bool Devuelto
{
    get { return devuelto; }
    set { devuelto = value; }
}
```

```
    public Prestamo(Usuario usuario, Libro libro, DateTime fechaPrestamo,
bool devuelto)
    {
        this.usuario = usuario;
        this.libro = libro;
        this.fechaPrestamo = fechaPrestamo;
        this.devuelto = devuelto;
    }
}

class Program
{
    static Libro[] libros = new Libro[100];
    static Usuario[] usuarios = new Usuario[100];
    static Prestamo[] prestamos = new Prestamo[100];

    static int totalLibros = 0;
    static int totalUsuarios = 0;
    static int totalPrestamos = 0;

    static void Main()
    {
        int opcion = 0;

        do
        {
            Console.Clear();
```

```
Console.WriteLine("==== GESTOR DE BIBLIOTECA ====");
Console.WriteLine("1. Registrar libro");
Console.WriteLine("2. Registrar usuario");
Console.WriteLine("3. Realizar préstamo");
Console.WriteLine("4. Devolver libro");
Console.WriteLine("5. Listar libros");
Console.WriteLine("6. Buscar libro");
Console.WriteLine("7. Reporte libros más prestados");
Console.WriteLine("8. Salir");
Console.Write("Seleccione una opción: ");

try
{
    opcion = int.Parse(Console.ReadLine());

    switch (opcion)
    {
        case 1:
            RegistrarLibro();
            break;
        case 2:
            RegistrarUsuario();
            break;
        case 3:
            RealizarPrestamo();
            break;
        case 4:
```

```
                DevolverLibro();
                break;
            case 5:
                ListarLibros();
                break;
            case 6:
                BuscarLibro();
                break;
            case 7:
                ReporteLibrosMasPrestados();
                break;
        }
    }
    catch
    {
        Console.WriteLine("Error: ingrese un número válido.");
    }

    Console.WriteLine("\nPresione una tecla para continuar...");
    Console.ReadKey();

  } while (opcion != 8);
}

static void RegistrarLibro()
{
  Console.Write("Título: ");
```

```
    string titulo = Console.ReadLine();

    Console.Write("Cantidad: ");
    int cantidad = int.Parse(Console.ReadLine());

    libros[totalLibros++] = new Libro(titulo, cantidad);

    Console.WriteLine("Libro registrado con éxito.");
}

static void RegistrarUsuario()
{
    Console.Write("Nombre: ");
    usuarios[totalUsuarios++] = new Usuario(Console.ReadLine());

    Console.WriteLine("Usuario registrado con éxito.");
}

static Usuario BuscarUsuario(string nombre)
{
    for (int i = 0; i < totalUsuarios; i++)
    {
        if (usuarios[i].Nombre.ToLower() == nombre.ToLower())
            return usuarios[i];
    }
    return null;
}
```

```
static Libro BuscarLibroObj(string titulo)
{
    for (int i = 0; i < totalLibros; i++)
    {
        if (libros[i].Titulo.ToLower() == titulo.ToLower())
            return libros[i];
    }
    return null;
}

static void RealizarPrestamo()
{
    Console.Write("Usuario: ");
    Usuario u = BuscarUsuario(Console.ReadLine());

    Console.Write("Libro: ");
    Libro l = BuscarLibroObj(Console.ReadLine());

    if (u == null || l == null)
    {
        Console.WriteLine("Usuario o libro no existe.");
        return;
    }

    if (l.Cantidad > 0)
    {
```

```
            prestamos[totalPrestamos++] = new Prestamo(u, l, DateTime.Now,
false);

            l.Cantidad--;
            l.Prestamos++;

            Console.WriteLine("Préstamo realizado con éxito.");
        }
        else
        {
            Console.WriteLine("Libro no disponible.");
        }
    }

    static void DevolverLibro()
    {
        Console.Write("Usuario: ");
        string nombre = Console.ReadLine();

        Console.Write("Libro: ");
        string titulo = Console.ReadLine();

        for (int i = 0; i < totalPrestamos; i++)
        {
            if (prestamos[i].Usuario.Nombre.ToLower() == nombre.ToLower() &&
                prestamos[i].Libro.Titulo.ToLower() == titulo.ToLower() &&
                !prestamos[i].Devuelto)
```

```
        {
            prestamos[i].Devuelto = true;
            prestamos[i].Libro.Cantidad++;

            int dias = (DateTime.Now - prestamos[i].FechaPrestamo).Days;

            if (dias > 7)
            {
                double multa = (dias - 7) * 0.50;
                Console.WriteLine("Libro devuelto con multa: $" +
multa.ToString("F2"));
            }
            else
            {
                Console.WriteLine("Libro devuelto sin multa.");
            }

            return;
        }
    }

    Console.WriteLine("No se encontró el préstamo.");
  }

  static void ListarLibros()
  {
    if (totalLibros == 0)
```

```
        {
            Console.WriteLine("No hay libros registrados.");
            return;
        }

        Console.WriteLine("\n=== LISTA DE LIBROS ===");
        for (int i = 0; i < totalLibros; i++)
        {
            Console.WriteLine(libros[i].Titulo + " | Cantidad: " + libros[i].Cantidad
+ " | Préstamos: " + libros[i].Prestamos);
        }
    }

    static void BuscarLibro()
    {
        Console.Write("Titulo: ");
        string t = Console.ReadLine();

        Libro l = BuscarLibroObj(t);

        if (l != null)
        {
            Console.WriteLine("Libro encontrado:");
            Console.WriteLine("Título: " + l.Titulo);
            Console.WriteLine("Cantidad: " + l.Cantidad);
            Console.WriteLine("Préstamos: " + l.Prestamos);
        }
```

```
    else
    {
        Console.WriteLine("Libro no existe.");
    }
}

static void ReporteLibrosMasPrestados()
{
    for (int i = 0; i < totalLibros - 1; i++)
    {
        for (int j = i + 1; j < totalLibros; j++)
        {
            if (libros[j].Prestamos > libros[i].Prestamos)
            {
                Libro aux = libros[i];
                libros[i] = libros[j];
                libros[j] = aux;
            }
        }
    }

    Console.WriteLine("\n=== LIBROS MÁS PRESTADOS ===");
    for (int i = 0; i < totalLibros; i++)
    {
        Console.WriteLine(libros[i].Titulo + " - " + libros[i].Prestamos);
    }
}}
```